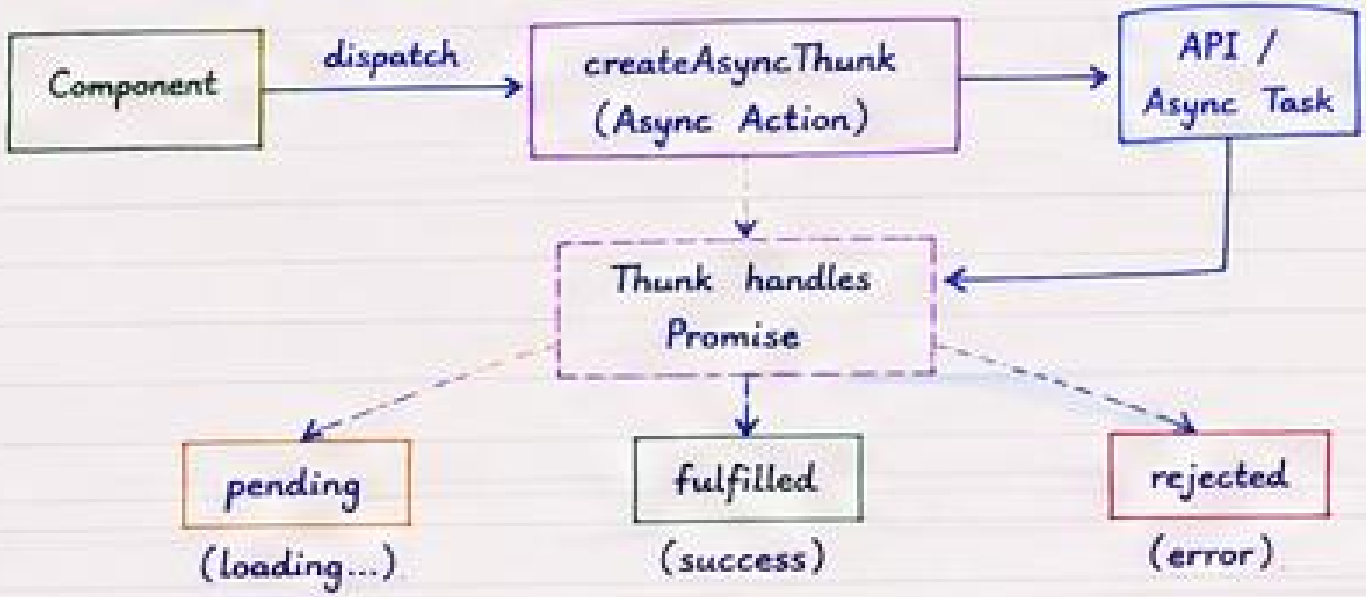
REDUX TOOLKIT (RTK) ASYNC - CHEAT SHEET ★

1. Why Async in Redux ?

- APIs, DB calls, timeouts → all are asynchronous.
- We need to handle → loading, success, error properly in the store.

2. The Hero - createAsyncThunk()

- Built-in function in RTK to handle async logic.
- Automatically generates 3 actions → _pending, fulfilled, rejected.



3. createAsyncThunk() - Syntax

```
const asyncThunk = createAsyncThunk(
  "sliceName/thunkName",     // Action type prefix
  async (arg, thunkAPI) => {   // Async logic
    return data;
  }
);
```

arg → any value you pass while dispatching.

thunkAPI → has helpers like rejectWithValue

4. Example - Fetch Users from API

```
// usersThunk.js
export const fetchUsers = createAsyncThunk(
  'users/fetchUsers',
  async (_, thunkAPI) => {
    try {
      const res = await fetch('https://jsonplaceholder.typicode.com/users');
      if (!res.ok) throw new Error("Failed to fetch");
      const data = await res.json();
      return data;
    } catch (err) {
      return thunkAPI.rejectWithValue(err.message);
    }
  }
);
```

We return data on success. On error → rejectWithValue() with message.

5. Handle in Slice - extraReducers

```
const usersSlice = createSlice({
  name: 'users',
  initialState: {
    users: [],
    loading: false,
    error: null
  },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchUsers.pending, (state) => {
        state.loading = true;
        state.error = null;
      })
      .addCase(fetchUsers.fulfilled, (state, action) => {
        state.loading = false;
        state.users = action.payload;
      })
      .addCase(fetchUsers.rejected, (state, action) => {
        state.loading = false;
        state.error = action.payload;
      });
```

3 Auto Actions

1. users/fetchUsers/pending
2. users/fetchUsers/fulfilled
3. users/fetchUsers/rejected

State Shape for Async

loading → boolean

users → []

error → string | null

6. Use in Component

```
// ExampleComponent.jsx
const MyComponent = () => {
  const dispatch = useDispatch();
  const { users, loading, error } = useSelector(
    (state) => state.users
  );

  useEffect(() => {
    dispatch(fetchUsers());
  }, [dispatch]);

  if (loading) return <p>Loading....</p>;
  if (error) return <p style={{color: 'red'}}>{error}</p>;

  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
};
```

dispatch the thunk like a normal action!

7. Dispatching with Parameters

```
// Pass parameter while dispatching
dispatch(fetchUserById(5));      // 5 → will be available in arg
dispatch(fetchUsers({ page: 1 })); // object also ok
```

8. Error Handling Best Practice

- Always use rejectWithValue() to send meaningful error.
- It will be available in action.payload in rejected case.

```
return thunkAPI.rejectWithValue('Something went wrong!');
```

9. rejectWithValue() vs throw new Error()

throw new Error()	rejectWithValue()
Sends error in action.error (less control)	Sends custom message in action.payload.
action.error.message will have the message.	You have full control over what error is stored.
Not recommended for expected errors (API errors).	Best practice for API error handling.

10. Other Useful ThunkAPI Helpers

- thunkAPI.dispatch → dispatch other actions
- thunkAPI.getState() → access current state
- thunkAPI.rejectWithValue(value) → handle errors
- thunkAPI.fulfillWithValue(value, meta?) → return custom fulfilled value

★ Async Flow Recap ★

dispatch(thunk) → pending → API Call →
fulfilled (success)
↓
rejected (error)

11. Complete Example in One Place

```
// usersSlice.js (Full Example)
import { createSlice, createAsyncThunk } from "@reduxjs/toolkit";

export const fetchUsers = createAsyncThunk(
  'users/fetchUsers',
  async (_, thunkAPI) => {
    try {
      const res = await fetch('https://jsonplaceholder.typicode.com/users');
      if (!res.ok) throw new Error('Failed to fetch');
      return await res.json();
    } catch (err) {
      return thunkAPI.rejectWithValue(err.message);
    }
  }
);
const usersSlice = createSlice({
  name: 'users',
  initialState: { users: [], loading: false, error: null },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchUsers.pending, (state) => {
        state.loading = true; state.error = null;
      })
      .addCase(fetchUsers.fulfilled, (state, action) => {
        state.loading = false; state.users = action.payload;
      })
      .addCase(fetchUsers.rejected, (state, action) => {
        state.loading = false; state.error = action.payload;
      });
  }
});
export default usersSlice.reducer;
```

Key Points

- ★ createAsyncThunk handles the async logic.
- ★ It creates 3 actions automatically.
- ★ Handle them in extraReducers.
- ★ Always manage loading & error in state.

Practice more with different APIs!

★ QUICK REVISION ★

1. Use createAsyncThunk for async logic.
2. It creates 3 actions → pending, fulfilled, rejected.
3. Handle in extraReducers using builder.
4. Manage loading, success data & error in state.
5. Use rejectWithValue() for meaningful errors.
6. Dispatch thunk like a normal action.
7. Access data in component using useSelector().
8. Keep state shape consistent.

STATE SHAPE TEMPLATE

```
{
    data: [],
    loading: false,
    error: null
}
```

Remember this always!